

Experiment 5

1. Aim

To design and simulate a **Common Bus System** using four 4-bit registers (D flip-flops with Write Enable and Reset) and four **2×1 Multiplexers** in **Logisim Evolution**, and to verify that any one of the four registers can selectively drive a shared output bus based on the MUX select lines.

2. Theory

A **Common Bus System** is a shared data pathway that connects multiple registers to a single set of output lines. At any given time, only one register is allowed to place its data on the bus; all others remain isolated. This prevents data collision and forms the backbone of data transfer in CPU register files and memory systems.

In this design, each of the **four 4-bit registers** (R0–R3) is built from D flip-flops with a **Write Enable (WE)** pin (to load data) and a **Reset (R)** pin (to clear the register). The Q output of each register feeds into a corresponding **MUX**. The four MUXes share common **select lines (S1, S0)**, which together determine which register's data appears on the bus output.

Bus Selection Truth Table:

S1	S0	Register on Bus
0	0	R0 (Register 0)
0	1	R1 (Register 1)
1	0	R2 (Register 2)
1	1	R3 (Register 3)

The clock signal is shared across all registers to synchronise data loading. Only the register whose **WE = 1** will latch new data on the clock edge; the rest retain their previous value.

3. Procedure

1. Open **Logisim Evolution** and create a new project file.
2. Build a **4-bit Register** sub-circuit using four D flip-flops. Connect a shared **Clock**, **Write Enable (WE)**, and **Reset (R)** to all four flip-flops. The 4-bit D input and 4-bit Q output form the register interface.
3. Place **four instances** of the register (R0, R1, R2, R3) in the main circuit. Connect the shared clock and individual 4-bit data input pins to each register.
4. Place **four 2×1 MUX** components (one per bit of the bus). Connect bit 0 of all four registers to the first MUX, bit 1 to the second, and so on.
5. Wire a common **2-bit select line (S1, S0)** to all four MUXes so they switch together.
6. Connect the MUX outputs to a shared **4-bit output bus** and attach a hex display for monitoring.

7. Set $WE = 1$ for a register and apply a clock pulse to load data into it; set $WE = 0$ for all others.
8. Toggle the select lines ($S1, S0$) through all four combinations (00 to 11) and verify the correct register's data appears on the bus output each time.
9. Take a screenshot of the completed and verified circuit.

4. Circuit Design / Implementation

The Common Bus System was implemented in **Logisim Evolution** with the following components:

- **4 × 4-bit Registers (R0–R3)** — each with D input (4-bit), Q output (4-bit), Write Enable (WE), Reset (R), and shared Clock.
- **4 × 2×1 MUXes** — one per bus bit, all sharing the same 2-bit select input ($S1, S0$) to simultaneously route the chosen register.
- **Common Bus (4-bit)** — the MUX outputs are merged into a single 4-bit bus connected to a hex display for real-time output.
- **Shared Clock** — a single clock input drives all registers; data is latched only in the register with $WE = 1$.

The select lines at the top of the circuit feed all four MUXes simultaneously. Register outputs (Q) feed horizontally into the MUX inputs, and the MUX outputs feed vertically into the common bus at the right, monitored by a hex display.

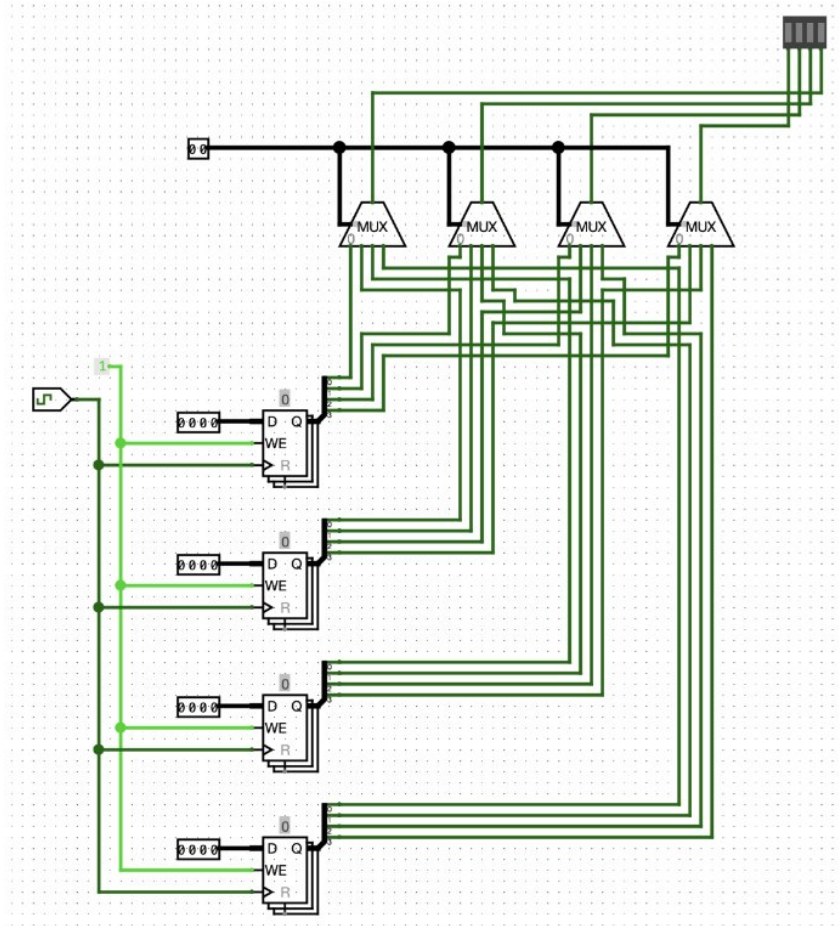


Figure 1: Common Bus System in Logisim Evolution — four 4-bit registers (R0–R3) with WE and Reset, four MUXes controlled by shared select lines (S1, S0), and a common 4-bit output bus with hex display.

5. Result

The Common Bus System was successfully designed and simulated in Logisim Evolution. Data was loaded into each of the four registers individually using the Write Enable and Clock signals. By varying the select lines (S1, S0) from 00 to 11, the output bus correctly displayed the contents of R0, R1, R2, and R3 respectively, confirming that only the selected register drives the bus at any given time.

6. Conclusion

This experiment demonstrated the design and working of a **Common Bus System** using registers and multiplexers in Logisim Evolution. The use of MUX select lines to control which register drives the shared bus reflects the fundamental principle of data routing used in CPU register files. The Write Enable and Reset controls provide precise management of register state, making this circuit a core building block of any digital processor architecture.